Reviewer Pack — Gromov 4-Point Hyperbolicity Gap Separates k-CLIQUE Monotone…

Entry 0420f93e5160 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 10:32:02 UTC

Gromov 4-Point Hyperbolicity Gap Separates k-CLIQUE Monotone DNFs from Random

Entry ID: `0420f93e5160` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 10:32:02 UTC

1. Conjecture

Field A (mathematical branch): Coarse geometry — Gromov δ-hyperbolicity of finite metric spaces (rarely applied to monotone complexity; appears mostly in geometric group theory and network science) **Field B** (complexity object): Monotone DNF representations of the k-CLIQUE indicator (Razborov monotone lower-bound setting, viewed as a query-to-DNF lifting target)

Statement:

> For a monotone k-uniform DNF F = T_1 ∨ … ∨ T_m on n Boolean variables (variables = edges of K_v where n = C(v,2)) with supports S_i ⊆ [n] of size k, define the symmetric-difference metric d_F(i,j) = |S_i △ S_j| and the normalized 4-point Gromov hyperbolicity *δ(F) = max_{i,j,k,l} ((M_1 − M_2)/(2·diam(F))) where M_1 ≥ M_2 ≥ M_3 are the sorted values of {d(i,j)+d(k,l), d(i,k)+d(j,l), d(i,l)+d(j,k)}. CONJECTURE: (i) For the CANONICAL k-CLIQUE DNF F_clique whose clauses enumerate every k-clique of K_v (k = ⌊√v⌋),* δ(F_clique) ≥ 1/4 − o(1) as v → ∞; (ii) For a uniformly random k-uniform monotone DNF F_rand with the same n, k, and number of clauses, E[δ*(F_rand)] ≤ C·v^{−1/2}·log v for an absolute constant C. Hence the ratio δ*(F_clique)/E[δ*(F_rand)] grows as Ω(√v / log v), giving a structural separator that submodularly lower-bounds any monotone DNF refinement of k-CLIQUE.

Rationale (proposer's reasoning):

> Gromov hyperbolicity quantifies how 'tree-like' a metric is on four-point configurations; the k-clique family in K_v has a rigid Johnson-scheme metric where almost every 4-tuple has two near-equal large pairwise sums (witnessing high δ*), while uniformly drawn k-subsets are with high probability metric-quasi-orthogonal (δ* concentrates near 0 by classical Hamming-cube concentration). If the gap persists, it identifies a coarse-geometric obstacle that any small monotone DNF cover of CLIQUE must encode, providing a query-like invariant ripe for Raz–McKenzie-style lifting into monotone circuit lower bounds without invoking sum-check or polynomial extension (so algebrization does not bite).

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `48d424369eef7b9c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across v ∈ {8,10,12,14,16,18,20} with 30 seeds per v and 5000 random 4-tuples per δ* estimate: δ*(F_clique) ≥ 0.20 for every v, mean-over-seeds δ*(F_rand) satisfies R_v = δ*(F_clique)/mean ≥ 0.3·√v/log v for ≥6 of 7 values of v, and no individual F_rand seed yields δ*(F_rand) ≥ 0.20.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Gromov hyperbolicity monotone circuit complexity clique` - `delta hyperbolicity Hamming metric DNF clause support` - `four-point condition random hypergraph k-clique lower bound`

Top relevant hits considered: - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [http://arxiv.org/abs/1307.4308v2] An Alternative Proof of the Exponential Monotone Complexity of the Clique Function - [http://arxiv.org/abs/2401.1780 Weighted-Hamming Metric for Parallel Channels - [http://arxiv.org/abs/math/0603539v2] Characterizations of metric trees and Gromov hyperbolic spaces - [http://arxiv.org/abs/1805.08177v4] A Polynomial Time Delta-Decomposition Algorithm for Positive DNFs - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=1.6s

5.1 Generated Python source

```python
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
```

```
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate
        for j in range(n):
            if i != j:
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gromov_hyperbolicity(F, v):
    n = v * (v - 1) // 2
    d_F = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            d_F[i][j] = abs(sum(1 for k in F if k & (1 << i)) - sum(1 for k in F if k & (1 << j)))
            d_F[j][i] = d_F[i][j]

    max_delta = 0
    for i, j, k, l in itertools.combinations(range(n), 4):
        M1 = d_F[i][j] + d_F[k][l]
        M2 = d_F[i][k] + d_F[j][l]
        M3 = d_F[i][l] + d_F[j][k]
        delta = (M1 - M2) / 2
        if delta > max_delta:
            max_delta = delta

    return max_delta

def run_trial(seed: int) -> dict:
    random.seed(seed)

    v_values = [8, 10, 12, 14, 16, 18, 20]
    results = []

    for v in v_values:
        n = v * (v - 1) // 2
        k = int(math.sqrt(v))

        # Build F_clique
        F_clique = [set(range(i, i+k)) for i in range(n-k+1)]
        delta_F_clique = gromov_hyperbolicity(F_clique, v)

        if delta_F_clique < 0.2:
            return {
                "metric_name": "delta_F_clique",
                "metric_value": delta_F_clique,
                "instances_tested": len(v_values),
```

```
                "conjecture_holds": False,
                "counterexample": "F_clique_delta_too_low"
            }

        # Build F_rand
        F_rand = []
        for _ in range(len(F_clique)):
            while True:
                S = set(random.sample(range(n), k))
                if all(S != T for T in F_rand):
                    F_rand.append(S)
                    break

        deltas_F_rand = [gromov_hyperbolicity(list(map(frozenset, F_rand)), v) for _ in range(5000
        mean_delta_F_rand = sum(deltas_F_rand) / len(deltas_F_rand)

        R_v = delta_F_clique / mean_delta_F_rand
        if R_v < 0.3 * math.sqrt(v) / math.log(v):
            return {
                "metric_name": "R_v",
                "metric_value": R_v,
                "instances_tested": len(v_values),
                "conjecture_holds": False,
                "counterexample": f"R_v_too_low for v={v}"
            }

        results.append({
            "delta_F_clique": delta_F_clique,
            "mean_delta_F_rand": mean_delta_F_rand,
            "R_v": R_v
        })

    return {
        "metric_name": "R_v",
        "metric_value": sum(result["R_v"] for result in results) / len(results),
        "instances_tested": len(v_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_R_v = sum(result["R_v"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_R_v:.6f} std=0.000000 support_fraction={support_fract
    elif support_fraction >= 0.8:
```

```
        print(f"RESULT: SUPPORTED mean={mean_R_v:.6f} std=0.000000 support_fraction={support_fract
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"R_v_too_low\" first_failing_seed={first_failing
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe3adfed.py", line 128, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe3adfed.py", line 75, in run_trial
    delta_F_clique = gromov_hyperbolicity(F_clique, v)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe3adfed.py", line 49, in gromov_hyperb
    d_F[i][j] = abs(sum(1 for k in F if k & (1 << i)) - sum(1 for k in F if k & (1 << j)))
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe3adfed.py", line 49, in <genexpr>
    d_F[i][j] = abs(sum(1 for k in F if k & (1 << i)) - sum(1 for k in F if k & (1 << j)))
                                        ~~^~~~~~~~~~
TypeError: unsupported operand type(s) for &: 'set' and 'int'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with a TypeError (bitwise & applied to a set instead of an int) before any data was produced, so neither the support nor falsification condition can be evaluated. | next: Fix the clause representation in gromov_hyperbolicity (encode each clause as a bitmask integer or compute d_F via |S_i △ S_j| on the actual support sets) and rerun the pre-registered sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	239373
2	preregistration	claude_max	opus	0	0	7258
3	novelty	claude_max	opus	0	0	4811
4	novelty	claude_max	opus	0	0	6995

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16086
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15395
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16749
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41367
9	judge	claude_max	opus	0	0	12604

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 360638 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/0420f93e5160.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0420f93e5160.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0420f93e5160.tar.gz` (if generated)